ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN A2 – MỐC A2.1

HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH – CO5085

PHÂN ĐOẠN NGỮ NGHĨA TRÊN ADE20K: SegFormer-B0 VÀ DeepLabV3+ (MobileNetV2)

Giảng viên phụ trách: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh

Lớp – Nhóm: CH01 – DL4DL

Sinh viên thực hiện: Lê Tiến Đạt – 2310653
Nguyễn Lê Thảo Ly – 2312010

Thành phố Hồ Chí Minh, Tháng 10/2026

MỤC LỤC

1 GIỚI THIỆU **6**
1.1 Bối cảnh nghiên cứu . . . 6
1.2 Phát biểu bài toán . . . 6
1.3 Mục tiêu nghiên cứu . . . 7
1.4 Phạm vi nghiên cứu . . . 8
1.5 Tài nguyên nghiên cứu . . . 9
1.6 Phân công và khai báo sử dụng công cụ AI . . . 9

2 Tập dữ liệu và chuẩn bị dữ liệu **10**
2.1 Giới thiệu tập dữ liệu . . . 10
2.2 Lý do lựa chọn tập dữ liệu . . . 11
2.3 Phân tích khám phá dữ liệu – EDA . . . 12
2.4 Phân chia tập dữ liệu . . . 12
2.5 Tiền xử lý và tăng cường dữ liệu . . . 13
2.6 Xây dựng DataLoader . . . 14
2.7 Điều khoản sử dụng dữ liệu . . . 15

3 PHƯƠNG PHÁP TRIỂN KHAI **16**
3.1 Tổng quan pipeline . . . 16
3.2 Biến thể 1: SegFormer-B0 . . . 17
3.3 Biến thể 2: DeepLabV3+ MobileNetV2 . . . 18
3.4 Biến thể khởi tạo: pretrained và huấn luyện từ đầu . . . 19

4 THIẾT KẾ THỰC NGHIỆM **20**
4.1 Cấu hình thí nghiệm . . . 20
4.1.1 Danh sách cấu hình . . . 20
4.1.2 Biến thay đổi và biến kiểm soát . . . 20
4.2 Thiết lập huấn luyện và siêu tham số . . . 21
4.2.1 Công thức huấn luyện của từng mô hình . . . 21

4.2.2 Dò learning rate . . . 22
4.2.3 Ngân sách 20 epoch . . . 22
4.3 Các chỉ số đánh giá . . . 23
4.3.1 Chỉ số đánh giá và phương pháp tính toán . . . 23
4.3.2 Quy trình đánh giá . . . 23
4.4 Nguyên tắc so sánh . . . 23
4.5 Môi trường thực nghiệm và khả năng tái lập . . . 24
4.5.1 Phần cứng và phần mềm . . . 24
4.5.2 Quản lý thí nghiệm và hướng dẫn tái lập . . . 25

5 PHÂN TÍCH BÀI BÁO KHOA HỌC 26
5.1 Giới thiệu bài báo . . . 26
5.2 Phương pháp được đề xuất . . . 27
5.2.1 Bộ mã hóa Mix Transformer (MiT) . . . 27
5.2.2 Bộ giải mã All-MLP . . . 27
5.2.3 Họ mô hình và độ phức tạp . . . 28
5.3 Thực nghiệm và kết quả của bài báo . . . 28
5.4 Hạn chế của bài báo . . . 30
5.5 Liên hệ với thí nghiệm của nhóm . . . 31

6 KẾT QUẢ THỰC NGHIỆM 32
6.1 Hiệu quả của các phương pháp . . . 32
6.1.1 Bảng và biểu đồ tổng hợp . . . 32
6.1.2 So sánh CNN và Transformer . . . 32
6.1.3 So sánh scratch và pretrained . . . 32
6.1.4 Ảnh hưởng của learning rate và độ ổn định giữa các seed . . . 32
6.1.5 IoU theo lớp và tần suất lớp . . . 32
6.2 Diễn biến quá trình huấn luyện . . . 32
6.2.1 Learning curves . . . 32
6.2.2 Kết quả dò learning rate . . . 32
6.3 Kết quả trực quan . . . 32
6.3.1 Ví dụ đầu ra và đối chiếu ground truth . . . 32
6.4 Chi phí tính toán . . . 33

7 PHÂN TÍCH VÀ THẢO LUẬN 34
7.1 So sánh các phương pháp hoặc biến thể . . . 34
7.1.1 Khác biệt về hiệu quả và các yếu tố ảnh hưởng . . . 34
7.1.2 Ưu điểm và hạn chế của từng phương pháp . . . 34

7.2 Phân tích các trường hợp thành công và thất bại 34
7.2.1 Trường hợp hoạt động tốt 34
7.2.2 Failure cases và nguyên nhân khả dĩ 34
7.3 Đối chiếu với bài báo khoa học 34
7.4 Hạn chế của thực nghiệm 34

8 KẾT LUẬN **36**
8.1 Tổng kết kết quả 36
8.2 Hướng phát triển 36

TÀI LIỆU THAM KHẢO **37**

A Kết quả thực nghiệm bổ sung **39**

B Kết quả trực quan bổ sung **40**

C Cấu hình và tái lập **41**
C.1 Dữ liệu 41
C.2 Cấu hình và lệnh chạy 41

DANH SÁCH BẢNG

1.1 Tài nguyên nghiên cứu (Research Resources) 9

2.1 Thông số của ADE20K SceneParsing. 11
2.2 Thống kê kích thước ảnh huấn luyện ADE20K (20 210 ảnh). 12
2.3 Cách chia dữ liệu của nhóm. 13
2.4 Tiền xử lý và tăng cường dữ liệu. 14

4.1 Các cấu hình huấn luyện chính thức (8 lượt chạy). 20
4.2 Công thức huấn luyện của hai mô hình. 21
4.3 Thời gian huấn luyện ước tính trên Tesla T4 (đo bằng benchmark; giờ/lượt là ngoại suy). Tổng cả gói khoảng 58,6 giờ T4. 22
4.4 Môi trường của các phép đo benchmark. 24

5.1 Họ SegFormer trên ADE20K theo Bảng 1a của bài báo (SS: một tỉ lệ; MS: nhiều tỉ lệ). 28
5.2 Bảng 1d của bài báo: bộ mã hóa CNN và Transformer với bộ giải mã MLP, ADE20K. 29
5.3 Các phương pháp thời gian thực trên ADE20K trong Bảng 2 của bài báo. 30

DANH SÁCH HÌNH VẼ

3.1 Pipeline tổng thể của thực nghiệm phân đoạn ngữ nghĩa. 16
3.2 SegFormer-B0: bộ mã hóa MiT bốn tầng và bộ giải mã All-MLP (số kênh của B0). 17
3.3 DeepLabV3+ MobileNetV2 (cấu hình mmsegmentation M-V2-D8); DS-conv là tích chập tách theo chiều sâu. 18

Chương 1

GIỚI THIỆU

1.1 Bối cảnh nghiên cứu

Phân đoạn ngữ nghĩa (semantic segmentation) là bài toán gán nhãn lớp cho từng điểm ảnh, và là nền tảng của nhiều ứng dụng như lái xe tự động, robot, hiểu cảnh trong nhà và ngoài trời, hay phân tích ảnh y khoa. Khác với phân loại ảnh ở bài tập lớn A1 (một nhãn cho cả ảnh), mô hình phải vừa nhận biết đối tượng vừa định vị chính xác ranh giới của chúng, nên chất lượng phụ thuộc đồng thời vào ngữ cảnh toàn ảnh và chi tiết ở độ phân giải cao.

Hướng tiếp cận chủ đạo xuất phát từ mạng tích chập hoàn toàn (Fully Convolutional Network, FCN) [6], biến một mạng phân loại thành bộ dự đoán dày. Các cải tiến sau đó tập trung vào việc mở rộng vùng tiếp nhận và thu thập ngữ cảnh, tiêu biểu là tích chập giãn (dilated/atrous convolution) và họ DeepLab với mô-đun ASPP [1, 2]. Gần đây, Transformer [11] được đưa vào thị giác máy tính qua ViT [4], rồi mở rộng sang dự đoán dày với SETR [15], PVT [12] và SegFormer [14]. SegFormer kết hợp bộ mã hóa Transformer phân cấp, không dùng positional encoding, với bộ giải mã chỉ gồm các lớp tuyến tính (All-MLP), và đạt độ chính xác cao với số tham số nhỏ trên các tập chuẩn như ADE20K [16].

Bài tập lớn A2 chọn bài toán này làm chủ đề, lấy SegFormer làm bài báo phân tích, và tái hiện phép so sánh giữa một mô hình Transformer nhẹ (SegFormer-B0) với một baseline CNN nhẹ (DeepLabV3+ với MobileNetV2) mà chính bài báo sử dụng, trong phạm vi tài nguyên tính toán của nhóm.

1.2 Phát biểu bài toán

Đầu vào. Một ảnh màu $I \in \mathbb{R}^{H \times W \times 3}$ có kích thước tùy ý (trong ADE20K, cạnh ngắn từ 96 đến 2 100 điểm ảnh).

Đầu ra. Một bản đồ nhãn $\hat{Y} \in \{0, 1, \ldots, 149\}^{H \times W}$ cùng kích thước với ảnh, trong đó mỗi

điểm ảnh được gán một trong $C = 150$ lớp đối tượng hoặc vật liệu của ADE20K SceneParsing.

Mục tiêu dự đoán. Mô hình tham số θ cho xác suất lớp tại từng điểm ảnh $p_\theta(c \mid I, u)$ và được huấn luyện để tối thiểu hóa entropy chéo trung bình trên các điểm ảnh có nhãn hợp lệ:

$$\mathcal{L}(\theta) = -\frac{1}{|\Omega|} \sum_{u \in \Omega} \log p_\theta(y_u \mid I, u), \qquad \Omega = \{u : \ y_u \neq \text{ignore}\}, \tag{1.1}$$

trong đó điểm ảnh thuộc nhóm "other objects" (giá trị nhãn 0) bị loại khỏi Ω. Chất lượng được đo bằng mIoU, Dice và độ chính xác điểm ảnh (mục 4.3).

Các khó khăn chính.

- *Số lớp lớn và phân bố đuôi dài:* 150 lớp, trong đó các lớp lớn (tường, bầu trời, sàn) chiếm phần lớn điểm ảnh còn nhiều lớp hiếm, nên mIoU bị kéo xuống bởi các lớp ít gặp.
- *Đối tượng nhiều kích thước và ranh giới mịn:* vật thể nhỏ, bị che khuất và ranh giới giữa các lớp đòi hỏi đặc trưng độ phân giải cao, nhưng đặc trưng ngữ nghĩa lại nằm ở các tầng sâu có độ phân giải thấp.
- *Chi phí tính toán:* dự đoán dày tốn kém hơn nhiều so với phân loại; việc huấn luyện trên ADE20K với lịch của bài báo (160 000 vòng lặp trên 8 GPU V100) vượt xa khả năng của nhóm.

Lý do cần khảo sát nhiều biến thể. Một con số mIoU riêng lẻ không cho biết nguồn cải thiện đến từ kiến trúc (Transformer so với CNN), từ việc khởi tạo (pretrained so với huấn luyện từ đầu) hay từ công thức huấn luyện. Do đó nhóm so sánh hai kiến trúc, mỗi kiến trúc ở hai chế độ khởi tạo, trên cùng dữ liệu, ngân sách và quy trình đánh giá.

1.3 Mục tiêu nghiên cứu

Theo đề bài, nhóm xây dựng pipeline huấn luyện và đánh giá chạy được, so sánh ít nhất hai phương pháp hoặc biến thể, phân tích một bài báo liên quan trực tiếp và đối chiếu với thực nghiệm của nhóm. Cụ thể:

1. Xây dựng pipeline phân đoạn ngữ nghĩa hoàn chỉnh trên ADE20K: dữ liệu và augmentation, huấn luyện có thể dừng và tiếp tục, đánh giá bằng mIoU, Dice và độ chính xác điểm ảnh từ ma trận nhầm lẫn.
2. So sánh **SegFormer-B0** (Transformer) với **DeepLabV3+ MobileNetV2** (CNN) ở hai chế độ khởi tạo (pretrained ImageNet-1K và huấn luyện từ đầu), nhằm tái hiện phép so sánh trong Bảng 2 của bài báo [14] dưới ngân sách nhỏ hơn.

3. Phân tích bài báo SegFormer: ý tưởng và kiến trúc, thí nghiệm, hạn chế; đối chiếu số liệu và kết luận của bài báo với kết quả của nhóm.

4. Đo và báo cáo chi phí (tham số, GMACs, tốc độ, bộ nhớ, giờ GPU) trên cùng một loại GPU để so sánh được.

5. Bảo đảm khả năng tái lập: seed, cấu hình, mã băm dữ liệu, checkpoint và notebook cho người chấm.

1.4 Phạm vi nghiên cứu

Trong phạm vi.

- Bài toán: phân đoạn ngữ nghĩa (không phải phân đoạn thể hiện hay toàn cảnh).
- Dữ liệu: ADE20K SceneParsing (ADEChallengeData2016), 150 lớp; huấn luyện trên 19 210 ảnh, chọn siêu tham số trên 1 000 ảnh tách từ tập huấn luyện, và chấm điểm một lần trên 2 000 ảnh validation chính thức (mục 2.4).
- Phương pháp: hai kiến trúc (SegFormer-B0, DeepLabV3+ MobileNetV2 bản mmsegmentation) × hai chế độ khởi tạo (pretrained, từ đầu), tức bốn cấu hình; ba seed cho mỗi cấu hình pretrained và một seed cho mỗi cấu hình từ đầu.
- Ngân sách: 20 epoch (24 000 vòng lặp, batch 16), bằng khoảng 15% lịch 160 000 vòng lặp của bài báo, giống nhau cho mọi lượt huấn luyện chính thức.
- Phân tích: mIoU/Dice/độ chính xác điểm ảnh, IoU theo lớp, nhầm lẫn giữa các lớp, chi phí tính toán, đối chiếu với bài báo.

Ngoài phạm vi.

- Các kiến trúc lớn hơn (SegFormer-B1 đến B5), các tập dữ liệu khác (Cityscapes, COCO-Stuff) và thí nghiệm độ bền Cityscapes-C của bài báo; nhóm chỉ phân tích các kết quả này ở phần bài báo.
- Lịch huấn luyện đầy đủ 160 000 vòng lặp, đánh giá đa tỉ lệ hoặc lật ảnh, và tối ưu hóa triển khai (TensorRT, lượng tử hóa).
- Tập test riêng của ban tổ chức ADE20K (nhãn không công khai).

Các nội dung mở rộng (kiểm tra độ bền theo độ phân giải, thay encoder bằng ResNet với cùng decoder MLP, demo) chỉ được thực hiện nếu còn thời gian sau khi hoàn thành phần chính.

1.5 Tài nguyên nghiên cứu

Bảng 1.1 tổng hợp các tài nguyên chính được sử dụng trong nghiên cứu.

Tài nguyên	**Thông tin và liên kết**
GitHub repository	`https://github.com/broistg/deep-learning-CO5085` (thư mục `A2`)
README	*Liên kết README hướng dẫn tái lập A2: bổ sung khi hoàn thành ở mốc A2.2.*
GitHub Pages	`https://broistg.github.io/deep-learning-CO5085/assignments/a2.html`
Google Colab	*Liên kết notebook thực nghiệm: bổ sung ở mốc A2.2.*
Báo cáo PDF	`https://broistg.github.io/deep-learning-CO5085/reports/assignment2.pdf`
Video demo / trình bày	*Bổ sung ở mốc A2.2.*
Bài báo phân tích	Xie et al., SegFormer, NeurIPS 2021: `https://arxiv.org/abs/2105.15203`; mã của tác giả: `https://github.com/NVlabs/SegFormer`
Tập dữ liệu	ADE20K SceneParsing: `https://ade20k.csail.mit.edu/`
Môi trường thực nghiệm	Google Colab, GPU Tesla T4 (15 GB), 2 nhân CPU; PyTorch 2.11.0 (CUDA 13.0), `transformers` 5.20.2, `timm` 1.0.30 (xem mục 4.5.1)

Bảng 1.1: Tài nguyên nghiên cứu (Research Resources)

1.6 Phân công và khai báo sử dụng công cụ AI

Nhóm gồm hai thành viên: Lê Tiến Đạt (2310653) và Nguyễn Lê Thảo Ly (2312010). *Bảng phân công chi tiết theo hạng mục (dữ liệu, huấn luyện, phân tích paper, báo cáo, trang web) sẽ được nhóm điền trước khi nộp bản cuối.*

Chương 2

TẬP DỮ LIỆU VÀ CHUẨN BỊ DỮ LIỆU

2.1 Giới thiệu tập dữ liệu

Nhóm sử dụng **ADE20K SceneParsing**, bản *ADEChallengeData2016*, do MIT CSAIL phát hành cho thử thách Scene Parsing [16]. Đây là tập ảnh cảnh trong nhà và ngoài trời đa dạng, mỗi ảnh chứa nhiều đối tượng và vật liệu thuộc nhiều lớp khác nhau, được gán nhãn ở mức điểm ảnh. Bảng 2.1 tóm tắt các thông số đã được nhóm kiểm tra trực tiếp trên tệp tải về.

Thông số	Giá trị
Nguồn	MIT CSAIL, benchmark Scene Parsing (`https://ade20k.csail.mit.edu/`); tệp `ADEChallengeData2016.zip`
Dung lượng	967 382 037 byte (922 MiB); mã sha256 của tệp nhóm đã tải: `7ff1be44...c977b29` (ghi đầy đủ ở Phụ lục C)
Số ảnh	20 210 ảnh huấn luyện; 2 000 ảnh validation (nhãn của tập test do ban tổ chức giữ kín nên không dùng)
Số lớp	150 lớp đối tượng/vật liệu. Trong ảnh nhãn PNG, giá trị $1 \dots 150$ là các lớp, giá trị 0 là "other objects" và bị bỏ qua khi đánh giá theo quy ước chính thức
Bố cục	`images/{training,validation}/ADE_*.jpg` và `annotations/{training,validation}/ADE_*.png`
Ánh xạ nhãn của nhóm	$0 \to 255$ (`ignore_index`), $1 \dots 150 \to 0 \dots 149$

Bảng 2.1: Thông số của ADE20K SceneParsing.

Việc điểm ảnh nhãn 0 bị loại khỏi loss và metric tương ứng với tham số `reduce_zero_label` của mmsegmentation mà bài báo sử dụng, nên số liệu của nhóm có thể đối chiếu trực tiếp với bài báo.

2.2 Lý do lựa chọn tập dữ liệu

- **Phù hợp đề bài:** ADE20K nằm trong danh sách tập dữ liệu được đề bài gợi ý cho bài toán phân đoạn, là tập công khai dùng được cho học tập, và đủ lớn (22 210 ảnh có nhãn công khai, 150 lớp) để việc đánh giá có ý nghĩa.
- **Đối chiếu được với bài báo:** ADE20K là benchmark chính của SegFormer, với số liệu của cả SegFormer-B0 và baseline DeepLabV3+ MobileNetV2 trong cùng một bảng (Bảng 2 của bài báo), nên nhóm có thể kiểm tra trực tiếp phép so sánh chính.
- **Độ khó:** 150 lớp, phân bố đuôi dài và ảnh rất đa dạng khiến bài toán khó hơn các tập nhỏ như Pascal VOC (21 lớp); mô hình nhẹ chỉ đạt khoảng 34–37 mIoU theo bài báo, nên có khoảng cách để phân biệt các phương pháp.
- **Khả thi về tài nguyên:** khoảng 20 000 ảnh huấn luyện cho phép huấn luyện hàng chục nghìn vòng lặp trên một GPU Colab; tệp tải về dài 922 MiB được tải trong khoảng 25 giây

khi nhóm đo, nên không cần dùng bộ nhớ đệm riêng trên Drive.

- **Điều khoản rõ ràng:** điều kiện sử dụng (nghiên cứu và giáo dục phi thương mại) được nêu công khai (mục 2.7), khác với một số tập thay thế như Pascal VOC, nơi điều khoản không được phát biểu rõ.

Nhóm đã cân nhắc Pascal VOC 2012 (kèm SBD) và loại vì tập nhỏ hơn đáng kể (1 464 ảnh huấn luyện chính thức, 1 449 ảnh validation), nhãn tập test không công khai, và số liệu của baseline MobileNetV2 trong bài báo không được báo cáo trên tập này.

2.3 Phân tích khám phá dữ liệu – EDA

Nhóm đã khảo sát toàn bộ 20 210 ảnh huấn luyện; các số liệu dưới đây đo trực tiếp trên tệp đã tải.

Đại lượng (điểm ảnh)	Nhỏ nhất	Trung vị	Lớn nhất
Chiều rộng	130	512	2 100
Chiều cao	96	480	2 100
Cạnh ngắn	96	473	2 100

Bảng 2.2: Thống kê kích thước ảnh huấn luyện ADE20K (20 210 ảnh).

Kích thước không đồng nhất. Không có kích thước ảnh cố định; 54,1% số ảnh có cạnh ngắn dưới 512 điểm ảnh. Vì vậy khi đánh giá (đưa cạnh ngắn về 512), phần lớn ảnh được phóng to thay vì thu nhỏ, và pipeline phải xử lý ảnh nhỏ hơn vùng cắt khi huấn luyện (mục 2.5).

Nhãn bị bỏ qua. Điểm ảnh mang nhãn 0 ("other objects") chiếm khoảng 9% tổng số điểm ảnh, ước lượng trên một mẫu ngẫu nhiên các ảnh huấn luyện.

Phân bố lớp. Với 150 lớp gồm cả đối tượng lớn (tường, bầu trời, sàn nhà) và các vật nhỏ hiếm gặp, phân bố tần suất lớp được kỳ vọng rất lệch (đuôi dài). Nhóm sẽ đo tần suất điểm ảnh theo từng lớp ở bước chuẩn bị dữ liệu và dùng nó để giải thích IoU theo lớp ở chương 6.

Hình phân bố tần suất lớp, ảnh và nhãn minh họa sẽ được bổ sung ở mốc A2.2 cùng với pipeline dữ liệu hoàn chỉnh.

2.4 Phân chia tập dữ liệu

Bộ dữ liệu ADE20K chỉ công bố nhãn cho tập huấn luyện và tập validation; nhãn tập test được ban tổ chức giữ kín và chỉ chấm qua máy chủ. Nhóm dùng cách chia ba phần để tránh chọn siêu tham số trên dữ liệu dùng để báo cáo cuối cùng (Bảng 2.3):

Tập	Số ảnh	Nguồn và mục đích
Train	19 210	Lấy ngẫu nhiên từ tập huấn luyện chính thức; dùng để cập nhật tham số
Dev (validation của nhóm)	1 000	Lấy ngẫu nhiên (seed 42) từ tập huấn luyện chính thức; dùng để chọn learning rate và vẽ đường học
Test	2 000	Toàn bộ tập validation chính thức của ADE20K; chấm điểm một lần ở cuối mỗi lượt chạy

Bảng 2.3: Cách chia dữ liệu của nhóm.

Danh sách ảnh của từng tập được lưu trong tệp `split.json` cùng mã sha256 và được dùng chung cho mọi lượt chạy. Tập test không tham gia chọn siêu tham số hay chọn checkpoint; kết quả được lấy ở vòng lặp cuối cùng. Cách chia này làm số ảnh huấn luyện ít hơn khoảng 5% so với tập huấn luyện đầy đủ mà bài báo dùng; đây là một sai khác được ghi nhận (mục 7.4).

Trong báo cáo, "validation (của nhóm)" chỉ tập dev tách từ tập huấn luyện, còn "test" chỉ 2 000 ảnh validation chính thức, để tránh nhầm lẫn với cách gọi trong tài liệu của ADE20K và bài báo (nơi "validation" là tập 2 000 ảnh dùng để báo cáo mIoU).

2.5 Tiền xử lý và tăng cường dữ liệu

Pipeline tăng cường dữ liệu bám theo cấu hình ADE20K của mmsegmentation mà bài báo sử dụng và được áp dụng giống nhau cho cả hai mô hình (Bảng 2.4). Mọi phép biến đổi hình học được áp dụng *đồng bộ* lên ảnh và nhãn: ảnh dùng nội suy song tuyến tính, nhãn dùng nội suy lân cận gần nhất để không tạo ra giá trị lớp mới.

Bước	Huấn luyện	Đánh giá
Đổi kích thước	Cạnh ngắn cơ sở 512, nhân ngẫu nhiên hệ số 0,5–2,0, giữ tỉ lệ khung hình	Cạnh ngắn 512 (cạnh dài tối đa 2 048), một tỉ lệ duy nhất
Cắt	Cắt ngẫu nhiên 512×512, không để một lớp chiếm quá 75% vùng cắt; ảnh nhỏ hơn vùng cắt được đệm (ảnh đệm giá trị 0, nhãn đệm 255)	Không cắt; suy luận trên toàn ảnh
Lật	Lật ngang với xác suất 0,5	Không
Quang học	Biến đổi độ sáng, tương phản, bão hòa, sắc độ (photometric distortion)	Không
Chuẩn hóa	Trừ trung bình và chia độ lệch chuẩn theo kênh của ImageNet; điểm ảnh nhãn 255 không tham gia loss	Như huấn luyện; điểm ảnh nhãn 255 không tham gia metric

Bảng 2.4: Tiền xử lý và tăng cường dữ liệu.

Phép chuẩn hóa dùng thống kê của ImageNet vì cả hai bộ mã hóa pretrained đều được huấn luyện trước trên ImageNet-1K với chuẩn hóa này; các cấu hình huấn luyện từ đầu dùng cùng chuẩn hóa để pipeline dữ liệu giống nhau. Mọi phép ngẫu nhiên dùng seed cố định theo từng lượt chạy để có thể lặp lại.

2.6 Xây dựng DataLoader

Nhóm tự cài đặt `Dataset` và `DataLoader` bằng PyTorch:

- **Đọc dữ liệu:** mỗi mẫu gồm ảnh JPEG và nhãn PNG tương ứng, đọc theo danh sách trong `split.json`; nhãn được ánh xạ $0 \to 255$, $1 \ldots 150 \to 0 \ldots 149$ ngay khi đọc.
- **Đầu vào/đầu ra:** sau augmentation, ảnh là tensor $3 \times 512 \times 512$ (chuẩn hóa) và nhãn là tensor nguyên 512×512 (giá trị $0 \ldots 149$ hoặc 255). Khi đánh giá, mỗi ảnh được xử lý riêng với kích thước thật (batch 1) vì kích thước ảnh khác nhau.
- **Batching:** batch 16 cho SegFormer; DeepLabV3+ dùng hai micro-batch 8 với tích lũy gradient để có batch hiệu dụng 16 (mục 4.2.1). Tập train được xáo trộn mỗi epoch.
- **Số worker:** Colab chỉ cấp 2 nhân CPU nên nhóm dùng 2 worker. Nhóm đã đo trước pipeline thật (đọc, giải mã, augmentation): 16,5 ms/mẫu trên một tiến trình và khoảng 80,7 ảnh/giây với 2 worker, đủ cho cả hai mô hình (SegFormer-B0 huấn luyện khoảng 19

ảnh/giây, DeepLabV3+ khoảng 13 ảnh/giây trên Tesla T4), nên DataLoader không phải điểm nghẽn.

Một epoch gồm $\lfloor 19\,210/16 \rfloor = 1\,200$ vòng lặp (bỏ batch cuối thiếu), nên 20 epoch tương ứng 24 000 vòng lặp.

2.7 Điều khoản sử dụng dữ liệu

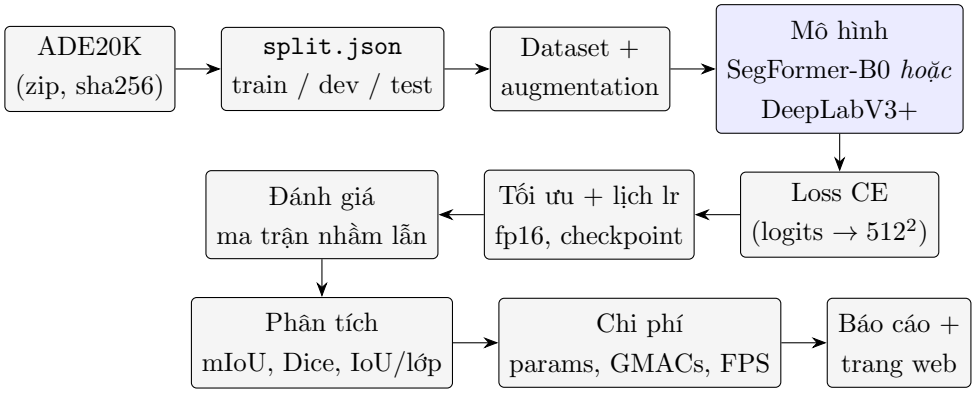
Theo điều khoản của ADE20K, dữ liệu chỉ được dùng cho mục đích nghiên cứu và giáo dục phi thương mại; MIT CSAIL không sở hữu bản quyền các ảnh và việc tiếp cận cho người khác phải qua cùng điều khoản. Vì vậy nhóm không phân phối lại ảnh hay bản đã xử lý: kho mã nguồn và các bản phát hành không chứa dữ liệu, mỗi thành viên tự tải tập dữ liệu và chỉ chia sẻ nội bộ giữa hai thành viên. Nhóm trích dẫn bài báo mô tả tập dữ liệu [16].

Chương 3

PHƯƠNG PHÁP TRIỂN KHAI

3.1 Tổng quan pipeline

Hình 3.1 mô tả luồng xử lý chung cho mọi cấu hình. Hai kiến trúc dùng chung toàn bộ phần dữ liệu, hàm mất mát, tính điểm và phân tích; chỉ phần mô hình (và công thức tối ưu tương ứng với mỗi mô hình) khác nhau.



Hình 3.1: Pipeline tổng thể của thực nghiệm phân đoạn ngữ nghĩa.

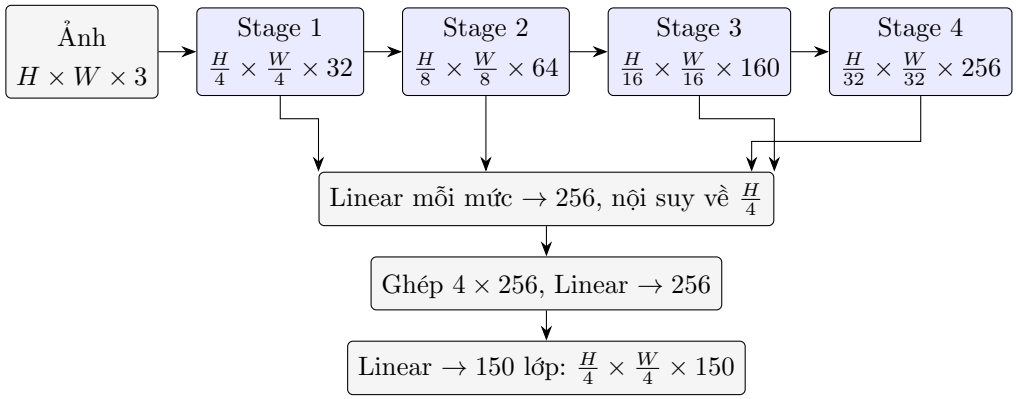
Quy trình thực hiện gồm ba giai đoạn:

1. **Chuẩn bị dữ liệu:** tải ADE20K, kiểm tra sha256, tạo `split.json`, đo EDA (chương 2).
2. **Chọn learning rate:** dò ba giá trị cho mỗi nhóm (kiến trúc × khởi tạo) trên 20% tập train trong 3 epoch, chọn theo mIoU trên tập dev (mục 4.2.2).
3. **Huấn luyện chính thức và đánh giá:** 20 epoch cho từng cấu hình với learning rate đã chọn; mỗi epoch ghi mIoU trên dev để vẽ đường học; cuối cùng chấm một lần trên tập test và đo chi phí.

Việc huấn luyện được ghi checkpoint sau mỗi epoch (mô hình, bộ tối ưu, scaler, lịch lr, trạng thái ngẫu nhiên) để có thể tiếp tục sau khi Colab ngắt kết nối mà vẫn giữ nguyên trạng thái huấn luyện.

3.2 Biến thể 1: SegFormer-B0

SegFormer [14] gồm một bộ mã hóa Transformer phân cấp (Mix Transformer, MiT) và một bộ giải mã chỉ gồm các lớp tuyến tính (All-MLP) (Hình 3.2). Phần này mô tả bản B0 mà nhóm sử dụng; phân tích đầy đủ ý tưởng của bài báo nằm ở chương 5.



Hình 3.2: SegFormer-B0: bộ mã hóa MiT bốn tầng và bộ giải mã All-MLP (số kênh của B0).

Bộ mã hóa MiT-B0. Bốn tầng cho đặc trưng ở độ phân giải 1/4, 1/8, 1/16, 1/32 ảnh gốc với số kênh $C_i = 32, 64, 160, 256$. Mỗi tầng gồm một lớp ghép patch chồng lấn (kernel 7, stride 4, đệm 3 ở tầng 1; kernel 3, stride 2, đệm 1 ở các tầng sau) và $L_i = 2$ khối Transformer; số head là $1, 2, 5, 8$, và tỉ lệ mở rộng của FFN là 4 ở cả bốn tầng theo cấu hình công bố `nvidia/mit-b0` (bảng siêu tham số trong phụ lục của bản arXiv ghi $8, 8, 4, 4$ cho hai tầng đầu, khác với cấu hình mã nguồn; nhóm dùng cấu hình của mã nguồn và số tham số tự đếm khớp bài báo). Self-attention hiệu quả rút gọn chuỗi khóa/giá trị theo tỉ lệ $R = [64, 16, 4, 1]$ (tương đương rút gọn $8, 4, 2, 1$ theo mỗi chiều không gian): với đầu vào 512×512, chuỗi ở tầng 1 có $N = 128^2 = 16\,384$ token nhưng chỉ có $N/64 = 256$ khóa, nên ma trận chú ý có $N \cdot N/R$ phần tử thay vì N^2. Khối FFN là Mix-FFN, thay positional encoding bằng một tích chập 3×3 (xem mục 5.2).

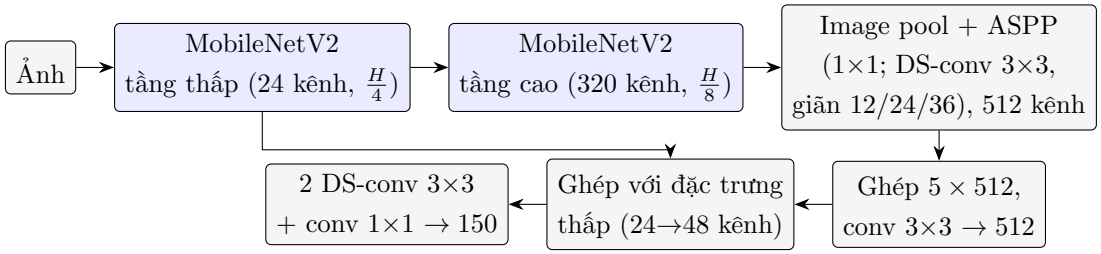
Bộ giải mã All-MLP. Mỗi đặc trưng F_i đi qua một lớp tuyến tính để đưa về $C = 256$ kênh, được nội suy lên độ phân giải 1/4, ghép lại, chiếu bằng một lớp tuyến tính $4C \to C$, rồi một lớp tuyến tính $C \to 150$ cho ra logits ở độ phân giải 1/4.

Cài đặt và chi phí. Nhóm dùng cài đặt `transformers` [13] của SegFormer và nạp trọng số bộ mã hóa `nvidia/mit-b0` (ImageNet-1K); bộ giải mã luôn được khởi tạo ngẫu nhiên. Mô hình 150 lớp do nhóm đếm có **3,753M tham số** và **8,39 GMACs** ở đầu vào $1 \times 3 \times 512 \times 512$, khớp với 3,8M và 8,4G của bài báo (Bảng 2). Với bộ mã hóa pretrained, nhóm *không bao giờ* khởi tạo

từ checkpoint `segformer-b0-finetuned-ade-512-512`, vì checkpoint đó đã được huấn luyện trên chính ADE20K và sẽ gây rò rỉ dữ liệu (mục 3.4).

3.3 Biến thể 2: DeepLabV3+ MobileNetV2

Baseline CNN là DeepLabV3+ [2] với bộ mã hóa MobileNetV2 [10], đúng mô hình "M-V2-D8" mà bài báo SegFormer dùng trong Bảng 2 (15,4M tham số, 69,4G FLOPs, 34,0 mIoU) và được cung cấp trong mmsegmentation [8]. Do thư viện mmsegmentation phụ thuộc nặng vào hệ sinh thái OpenMMLab, nhóm tự cài đặt lại mô hình bằng PyTorch thuần và kiểm tra khớp số tham số và GMACs với bài báo (Hình 3.3).



Hình 3.3: DeepLabV3+ MobileNetV2 (cấu hình mmsegmentation M-V2-D8); DS-conv là tích chập tách theo chiều sâu.

Bộ mã hóa. MobileNetV2 được biến đổi để đầu ra có output stride 8: các stride là $(1, 2, 2, 1, 1, 1, 1)$ và độ giãn $(1, 1, 1, 2, 2, 4, 4)$ ở bảy nhóm khối, cho đặc trưng 320 kênh ở độ phân giải 1/8 và đặc trưng mức thấp 24 kênh ở 1/4.

Bộ giải mã. Đặc trưng 320 kênh đi qua nhánh gộp toàn cục (image pooling) và mô-đun ASPP gồm tích chập 1×1 và ba tích chập tách theo chiều sâu 3×3 với độ giãn $12, 24, 36$; tất cả có 512 kênh. Năm nhánh được ghép (5×512) và chiếu bằng một tích chập 3×3 đầy đủ về 512 kênh — phần chiếm nhiều tham số nhất (khoảng 11,8M). Sau đó đặc trưng được nội suy lên 1/4, ghép với đặc trưng mức thấp đã giảm từ 24 xuống 48 kênh, đi qua hai tích chập tách theo chiều sâu 3×3 và một tích chập 1×1 cho 150 lớp. Một đầu phụ trợ FCN trên tầng 96 kênh chỉ dùng khi huấn luyện (trọng số loss 0,4).

Chi phí. Mô hình nhóm cài đặt có **15,345M tham số** (15,085M không tính đầu phụ trợ) và **69,44 GMACs** ở $1 \times 3 \times 512 \times 512$ (đo bằng `fvcore`), khớp với 15,4M và 69,4G trong bài báo. Nhóm *không* dùng bản DeepLabV3+ trong thư viện `segmentation_models_pytorch`: bản đó chỉ có 4,4M tham số và 6,6 GMACs (tích chập chiếu 1×1 thay cho tích chập 3×3 đầy đủ) nên không còn là baseline của bài báo. Bộ mã hóa pretrained dùng trọng số ImageNet-1K của torchvision (`IMAGENET1K_V1`), khác trọng số của mmcls mà mmsegmentation dùng (xem hạn chế ở mục 7.4).

Khác biệt chính so với SegFormer-B0. Với số tham số gấp khoảng 4 lần và chi phí tính toán gấp khoảng 8 lần, DeepLabV3+ dựa vào tích chập giãn và ASPP để mở rộng vùng tiếp nhận,

trong khi SegFormer có vùng tiếp nhận toàn cục ngay từ self-attention ở các tầng sâu và chỉ cần bộ giải mã tuyến tính. Đây chính là điểm mà bài báo lập luận (mục 5.2) và nhóm sẽ kiểm tra bằng thực nghiệm.

3.4 Biến thể khởi tạo: pretrained và huấn luyện từ đầu

Mỗi kiến trúc được huấn luyện ở hai chế độ khởi tạo bộ mã hóa; bộ giải mã luôn khởi tạo ngẫu nhiên.

- **Pretrained:** bộ mã hóa nạp trọng số ImageNet-1K (SegFormer: `nvidia/mit-b0`; DeepLabV3+: MobileNetV2 của torchvision). Đây là thiết lập của bài báo.
- **Từ đầu (scratch):** bộ mã hóa khởi tạo ngẫu nhiên, nhằm đo mức phụ thuộc vào pretrain khi chỉ có khoảng 19 000 ảnh huấn luyện.

Quy tắc chống rò rỉ. Không bao giờ khởi tạo từ checkpoint đã được huấn luyện trên ADE20K (ví dụ `nvidia/segformer-b0-finetuned-ade-512-512`) hay bất kỳ checkpoint phân đoạn nào khác: những trọng số này đã thấy dữ liệu ADE20K, kể cả ảnh thuộc tập test của nhóm. Checkpoint công bố của tác giả chỉ được dùng ở chế độ suy luận để kiểm chứng quy trình đánh giá (mục 4.3.2). Các trọng số của NVIDIA đi kèm giấy phép riêng; nhóm chỉ công bố trọng số do chính nhóm huấn luyện, có ghi nguồn.

Chương 4

THIẾT KẾ THỰC NGHIỆM

4.1 Cấu hình thí nghiệm

4.1.1 Danh sách cấu hình

Hai kiến trúc nhân hai chế độ khởi tạo cho bốn cấu hình (Bảng 4.1). Các cấu hình pretrained được chạy với ba seed để ước lượng độ ổn định; các cấu hình từ đầu chỉ chạy một seed vì chỉ nhằm cho thấy mức phụ thuộc vào pretrain. Ngoài ra, mỗi cấu hình có một đợt dò learning rate ngắn (mục 4.2.2), tổng cộng 12 lượt dò và 8 lượt huấn luyện đầy đủ.

Kiến trúc	Khởi tạo encoder	Seed	Mục đích
SegFormer-B0	ImageNet-1K	0, 1, 2	Tái hiện phép so sánh của bài báo
SegFormer-B0	Ngẫu nhiên	0	Mức phụ thuộc vào pretrain
DeepLabV3+ MobileNetV2	ImageNet-1K	0, 1, 2	Baseline CNN của bài báo
DeepLabV3+ MobileNetV2	Ngẫu nhiên	0	Mức phụ thuộc vào pretrain

Bảng 4.1: Các cấu hình huấn luyện chính thức (8 lượt chạy).

4.1.2 Biến thay đổi và biến kiểm soát

Biến thay đổi: kiến trúc (Transformer so với CNN) và chế độ khởi tạo (pretrained so với từ đầu); learning rate được chọn riêng cho từng cấu hình bằng cùng một quy trình dò.

Biến giữ cố định: dữ liệu và cách chia (`split.json`); pipeline augmentation; kích thước batch hiệu dụng (16); ngân sách huấn luyện (20 epoch, 24 000 vòng lặp); hàm mất mát chính (entropy chéo, `ignore_index` 255, trên logits nội suy lên 512×512); độ chính xác fp16; quy trình đánh giá; loại GPU (Tesla T4).

Lưu ý về công thức tối ưu. Mỗi mô hình dùng công thức huấn luyện chính thức của nó (SegFormer: AdamW; DeepLabV3+: SGD) thay vì ép dùng chung một bộ tối ưu, vì mục tiêu là tái hiện phép so sánh của bài báo giữa hai mô hình ở trạng thái được tinh chỉnh của mỗi bên. Hệ quả là kiến trúc và công thức tối ưu bị "lẫn" vào nhau; điều này được nêu như một hạn chế khi phân tích kết quả.

4.2 Thiết lập huấn luyện và siêu tham số

4.2.1 Công thức huấn luyện của từng mô hình

Công thức dưới đây lấy từ cấu hình chính thức của mỗi mô hình: SegFormer-B0 theo cấu hình ADE20K của tác giả (kho NVlabs/SegFormer), DeepLabV3+ theo lịch mặc định `schedule_160k` của mmsegmentation đã tạo ra con số 34,0 mIoU của bài báo.

Thành phần	SegFormer-B0	DeepLabV3+ MobileNetV2
Bộ tối ưu	AdamW, $\beta = (0{,}9; 0{,}999)$, weight decay 0,01 (không áp dụng cho norm và thành phần vị trí)	SGD, momentum 0,9, weight decay $5 \cdot 10^{-4}$
Learning rate	Cơ sở $6 \cdot 10^{-5}$ (chọn bằng dò), decoder nhân 10	Cơ sở 0,01 (chọn bằng dò), không nhân riêng
Lịch lr	Poly (mũ 1,0), lr cuối 0; warmup tuyến tính 6,25% số vòng lặp (1 500 trên 24 000)	Poly (mũ 0,9), lr cuối 10^{-4}; không warmup
Hàm mất mát	Entropy chéo	Entropy chéo; thêm loss đầu phụ trợ trọng số 0,4
Batch	16	16 (hai micro-batch 8, tích lũy gradient)
Độ chính xác	fp16 (autocast) trên Tesla T4	
Độ dài	20 epoch = 24 000 vòng lặp	
Đánh giá	Vòng lặp cuối cùng, không chọn checkpoint theo tập test	

Bảng 4.2: Công thức huấn luyện của hai mô hình.

Hai điểm khác bản gốc được ghi nhận: (i) bản gốc dùng batch 16 trên 4 GPU với SyncBN, còn nhóm dùng tích lũy gradient 2×8 trên một GPU nên thống kê BatchNorm tính trên 8 ảnh; (ii) nhóm dùng fp16 để vừa bộ nhớ và tăng tốc, còn bài báo không nêu độ chính xác. Hàm mất mát

cho cả hai mô hình được tính trên logits được nội suy song tuyến tính lên 512×512 như trong bài báo; cách này tốn bộ nhớ hơn (khoảng 11 GB cho SegFormer ở batch 16) nhưng tránh làm khác đi bài toán tối ưu so với bài báo.

4.2.2 Dò learning rate

Learning rate được chọn cho từng nhóm (kiến trúc × khởi tạo) bằng cách huấn luyện ngắn trên 20% tập train (3 842 ảnh), 3 epoch (720 vòng lặp), seed 0, và chọn giá trị cho mIoU cao nhất trên tập dev (nếu bằng nhau, chọn loss dev thấp hơn). Nếu giá trị tốt nhất nằm ở biên lưới, lưới được mở thêm một bước. Lưới ban đầu:

- SegFormer pretrained: $\{3{\cdot}10^{-5},\ 6{\cdot}10^{-5},\ 1{,}2{\cdot}10^{-4}\}$; SegFormer từ đầu: $\{2{\cdot}10^{-4},\ 6{\cdot}10^{-4},\ 1{,}8{\cdot}10^{-3}\}$.
- DeepLabV3+ pretrained: $\{0{,}005,\ 0{,}01,\ 0{,}02\}$; DeepLabV3+ từ đầu: $\{0{,}01,\ 0{,}03,\ 0{,}1\}$.

Bốn nhóm × ba giá trị cho 12 lượt dò. Hệ số nhân 10 cho learning rate của decoder (SegFormer) được giữ cố định. Việc dò chỉ dùng tập dev, không dùng tập test.

4.2.3 Ngân sách 20 epoch

Bài báo huấn luyện 160 000 vòng lặp (khoảng 133 epoch ở batch 16) trên 8 GPU V100 và 40 000 vòng lặp cho các thí nghiệm ablation. Nhóm chọn **20 epoch** = 24 000 **vòng lặp**, bằng 15% lịch đầy đủ và 60% lịch ablation, giống nhau cho mọi lượt chạy chính thức. Quyết định này dựa trên các phép đo trên Tesla T4 (Bảng 4.3): SegFormer-B0 mất khoảng 16,9 phút/epoch (11 GB bộ nhớ ở batch 16), DeepLabV3+ khoảng 25,1 phút/epoch (batch 8, 9,5 GB). Tổng khoảng 58,6 giờ T4 cho 8 lượt chạy chính thức và 12 lượt dò; nhóm không có khả năng chạy lịch đầy đủ của bài báo. Hệ quả cần nêu rõ ngay từ đầu: mIoU tuyệt đối sẽ thấp hơn bài báo, đặc biệt ở các lượt huấn luyện từ đầu chưa hội tụ; điều cần kiểm tra là thứ tự và khoảng cách tương đối giữa các cấu hình.

	Phút/ epoch	**Giờ/lượt 20 epoch**	**Giờ/lượt dò (3 epoch, 20% dữ liệu)**
SegFormer-B0	16,9	5,6	0,18
DeepLabV3+ MobileNetV2	25,1	8,4	0,25

Bảng 4.3: Thời gian huấn luyện ước tính trên Tesla T4 (đo bằng benchmark; giờ/lượt là ngoại suy). Tổng cả gói khoảng 58,6 giờ T4.

4.3 Các chỉ số đánh giá

4.3.1 Chỉ số đánh giá và phương pháp tính toán

Mọi chỉ số đều được tính từ *một ma trận nhầm lẫn* 150×150 *cộng dồn trên toàn bộ tập test* (không lấy trung bình theo từng ảnh), bỏ qua điểm ảnh nhãn 255. Với lớp c, gọi TP_c, FP_c, FN_c lần lượt là số điểm ảnh dự đoán đúng, dự đoán thừa và dự đoán thiếu:

$$\mathrm{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \qquad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c, \tag{4.1}$$

$$\mathrm{Dice}_c = \frac{2\,TP_c}{2\,TP_c + FP_c + FN_c}, \qquad \mathrm{PixAcc} = \frac{\sum_c TP_c}{\sum_c (TP_c + FN_c)}. \tag{4.2}$$

mIoU là chỉ số chính (cũng là chỉ số của bài báo); nhóm báo cáo thêm Dice trung bình (theo yêu cầu của đề bài cho bài toán phân đoạn), độ chính xác điểm ảnh và IoU theo từng lớp. Lớp không xuất hiện trong nhãn lẫn dự đoán có mẫu số bằng không; số lượng lớp như vậy được đếm và báo cáo thay vì bị bỏ qua âm thầm.

4.3.2 Quy trình đánh giá

- **Tập dùng:** tập dev (1 000 ảnh) được tính mIoU sau mỗi epoch để vẽ đường học và chọn learning rate; tập test (2 000 ảnh validation chính thức) chỉ được chấm *một lần* ở vòng lặp cuối của mỗi lượt chạy.
- **Suy luận:** một tỉ lệ duy nhất (cạnh ngắn 512, giữ tỉ lệ khung hình, cạnh dài tối đa 2 048), không lật ảnh, không ghép nhiều tỉ lệ; logits được nội suy song tuyến tính về kích thước nhãn gốc.
- **Kiểm chứng đánh giá:** để chắc rằng mã đánh giá đúng, nhóm chạy checkpoint SegFormer-B0 do tác giả công bố (đã huấn luyện trên ADE20K) chỉ ở chế độ suy luận trên 2 000 ảnh validation, đạt **36,70 mIoU** và 77,43% độ chính xác điểm ảnh, sát với 37,4 mIoU của bài báo (chênh 0,7 điểm). Không có lớp nào vắng mặt đồng thời trong nhãn và dự đoán. Checkpoint này không dùng cho huấn luyện.

Chênh lệch nhỏ còn lại có thể đến từ khác biệt cài đặt nội suy và tiền xử lý so với mmsegmentation.

4.4 Nguyên tắc so sánh

Để phép so sánh có ý nghĩa, nhóm tuân theo các nguyên tắc sau:

1. **Cùng dữ liệu, cùng ngân sách, cùng đánh giá:** mọi cấu hình dùng chung `split.json`, pipeline augmentation, 24 000 vòng lặp ở batch 16 và quy trình đánh giá ở mục 4.3.2.

2. **Học tập công bằng ở mức siêu tham số:** learning rate của mỗi cấu hình được chọn bằng cùng một quy trình dò trên tập dev, không trên tập test.

3. **Nhiều seed:** các cấu hình pretrained chạy ba seed (0, 1, 2) và được báo cáo dưới dạng trung bình ± độ lệch chuẩn mẫu; một chênh lệch nhỏ hơn độ lệch chuẩn giữa các seed không được xem là có ý nghĩa. Cấu hình từ đầu chỉ có một seed nên không có độ lệch chuẩn và chỉ được dùng để so sánh thô.

4. **Không chọn checkpoint theo test:** kết quả là điểm ở vòng lặp cuối cùng; tập test được chấm một lần.

5. **Chi phí đo trên cùng phần cứng:** tham số, GMACs ở $1 \times 3 \times 512 \times 512$, tốc độ huấn luyện và suy luận, bộ nhớ đỉnh được đo trên cùng một loại GPU (Tesla T4).

6. **Công bố rõ các sai khác:** các khác biệt so với bài báo (độ dài lịch, kích thước tập train, thống kê BatchNorm, trọng số ImageNet, fp16) được liệt kê ở chương 7.

4.5 Môi trường thực nghiệm và khả năng tái lập

4.5.1 Phần cứng và phần mềm

Mọi lượt chạy A2 thực hiện trên Google Colab với cùng một loại GPU để thời gian và bộ nhớ so sánh được (Bảng 4.4). Các phiên bản dưới đây là của phiên đo benchmark; phiên bản dùng cho các lượt chạy chính thức sẽ được ghi trong từng tệp kết quả.

Thành phần	Giá trị
GPU	NVIDIA Tesla T4, 15 GB
CPU	2 nhân (Colab)
PyTorch	2.11.0 (CUDA 13.0)
`transformers`	5.20.2
`timm`	1.0.30
`segmentation_models_pytorch`	0.5.0 (chỉ dùng cho benchmark, không dùng cho mô hình chính)

Bảng 4.4: Môi trường của các phép đo benchmark.

4.5.2 Quản lý thí nghiệm và hướng dẫn tái lập

- **Seed và cấu hình:** mỗi lượt chạy có một tệp cấu hình ghi đầy đủ siêu tham số, seed và mã băm của `split.json`; seed điều khiển khởi tạo, thứ tự dữ liệu và augmentation.
- **Checkpoint và tiếp tục:** checkpoint sau mỗi epoch gồm mô hình, bộ tối ưu, scaler, lịch lr và trạng thái ngẫu nhiên; khi Colab ngắt kết nối, lượt chạy được tiếp tục từ epoch cuối. Hàng đợi các lượt chạy dùng cơ chế dùng chung với bài tập A1 và lưu trên một thư mục Drive chung của hai thành viên.
- **Kết quả:** mỗi lượt chạy ghi một tệp JSON (cấu hình, lịch sử, kết quả cuối, thời gian, môi trường) và ma trận nhầm lẫn; các bảng và hình trong báo cáo được sinh từ những tệp này.
- **Notebook tái lập:** một notebook cho phép hoặc nạp lại kết quả đã lưu để sinh lại bảng và hình, hoặc chạy lại toàn bộ thí nghiệm (kèm cảnh báo thời gian).

Cấu trúc thư mục mã nguồn, liên kết GitHub/Colab và hướng dẫn chạy chi tiết sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), khi mã huấn luyện được đưa lên kho.

Chương 5

PHÂN TÍCH BÀI BÁO KHOA HỌC

5.1 Giới thiệu bài báo

Bài báo: Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, Ping Luo. *"SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers"*. Advances in Neural Information Processing Systems 34 (NeurIPS 2021). arXiv:2105.15203. Mã nguồn của tác giả: `https://github.com/NVlabs/SegFormer` [14].

Vấn đề nghiên cứu. Các mô hình phân đoạn dựa trên Transformer thời điểm đó đi theo hai hướng: dùng nguyên bộ mã hóa ViT (như SETR [15]), vốn chỉ cho đặc trưng một tỉ lệ ở độ phân giải thấp, đòi hỏi positional embedding có kích thước cố định và bộ giải mã nặng; hoặc thiết kế lại bộ mã hóa phân cấp (như PVT [12]) nhưng vẫn giữ các bộ giải mã phức tạp. Câu hỏi của bài báo là liệu có thể đạt độ chính xác cao mà vẫn đơn giản và hiệu quả bằng cách vừa thiết kế lại bộ mã hóa, vừa thu gọn bộ giải mã, đồng thời giữ được độ bền khi độ phân giải lúc kiểm tra khác lúc huấn luyện.

Đóng góp chính (diễn đạt lại). (i) Một bộ mã hóa Transformer phân cấp *không cần positional encoding*; (ii) một bộ giải mã chỉ gồm các lớp tuyến tính (All-MLP) gộp đặc trưng nhiều tầng; (iii) họ mô hình B0–B5 cho các mức đánh đổi khác nhau giữa độ chính xác và chi phí, với kết quả tốt nhất trên ADE20K và Cityscapes tại thời điểm công bố cùng độ bền cao hơn trước các nhiễu như mờ, nhiễu hạt và thời tiết.

Lý do chọn và liên hệ với pipeline. SegFormer là bài báo có ảnh hưởng, được nhận tại hội nghị hàng đầu (NeurIPS), liên quan trực tiếp đến phân đoạn ngữ nghĩa và được đề bài nêu trong danh sách phương pháp. Bài báo báo cáo ngay trên ADE20K cả SegFormer-B0 lẫn baseline DeepLabV3+ MobileNetV2 (Bảng 2), nên thí nghiệm của nhóm (mục 5.5) tái hiện trực tiếp một phép so sánh có thật trong bài báo, với cùng tập dữ liệu, cùng hai mô hình nhẹ và cùng metric.

5.2 Phương pháp được đề xuất

5.2.1 Bộ mã hóa Mix Transformer (MiT)

Ý tưởng cốt lõi là một Transformer *phân cấp* giống như cách CNN thu nhỏ dần độ phân giải: một ảnh $H \times W \times 3$ được chia thành patch 4×4 (nhỏ hơn nhiều so với 16×16 của ViT vì phân đoạn cần chi tiết mịn) và qua bốn tầng, cho ra đặc trưng F_i có độ phân giải $\frac{H}{2^{i+1}} \times \frac{W}{2^{i+1}}$ với số kênh C_i tăng dần ($i = 1, \ldots, 4$), tức tỉ lệ $1/4, 1/8, 1/16, 1/32$. Bốn thành phần của bộ mã hóa:

(1) Ghép patch chồng lấn (overlapped patch merging). Việc thu nhỏ giữa các tầng dùng tích chập với kernel K, stride S, đệm P ($K = 7, S = 4, P = 3$ ở tầng đầu; $K = 3, S = 2, P = 1$ ở các tầng sau) thay vì ghép các patch rời nhau, để giữ tính liên tục cục bộ quanh ranh giới patch.

(2) Self-attention hiệu quả. Ở độ phân giải cao, chuỗi token có độ dài $N = H \times W$ rất lớn nên chi phí $O(N^2)$ của self-attention

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_{head}}}\right) V \tag{5.1}$$

trở thành nút thắt. Theo PVT [12], chuỗi khóa K được rút gọn: reshape từ $N \times C$ thành $\frac{N}{R} \times (CR)$ rồi chiếu tuyến tính $CR \to C$, nên chuỗi khóa/giá trị chỉ còn N/R phần tử và chi phí giảm còn $O(N^2/R)$. Bài báo chọn $R = [64, 16, 4, 1]$ từ tầng 1 đến tầng 4.

(3) Mix-FFN thay positional encoding. ViT cộng một positional embedding có kích thước cố định, nên phải nội suy khi độ phân giải kiểm tra khác độ phân giải huấn luyện và độ chính xác giảm. Tác giả thay bằng việc chèn một tích chập 3×3 (tách theo chiều sâu) vào giữa hai lớp tuyến tính của FFN:

$$x_{out} = \text{MLP}(\text{GELU}(\text{Conv}_{3\times3}(\text{MLP}(x_{in})))) + x_{in}. \tag{5.2}$$

Giả thiết là hiệu ứng đệm số không của tích chập đủ để làm lộ thông tin vị trí tuyệt đối, nên không cần positional encoding riêng; vì tích chập không phụ thuộc kích thước đầu vào, mô hình thích nghi được với độ phân giải bất kỳ.

(4) Khối Transformer. Mỗi tầng xếp L_i khối gồm self-attention hiệu quả và Mix-FFN.

5.2.2 Bộ giải mã All-MLP

Bộ giải mã chỉ gồm các lớp tuyến tính, theo bốn bước (công thức gốc (4)): (a) đưa mỗi F_i về cùng số kênh C bằng $\text{Linear}(C_i, C)$; (b) nội suy tất cả lên độ phân giải $1/4$; (c) ghép bốn đặc trưng rồi trộn bằng $\text{Linear}(4C, C)$; (d) dự đoán mặt nạ $\frac{H}{4} \times \frac{W}{4} \times N_{cls}$ bằng $\text{Linear}(C, N_{cls})$. Hàm mất mát là entropy chéo trên mặt nạ được nội suy lên kích thước nhãn; bài báo không dùng OHEM, loss phụ trợ hay loss cân bằng lớp.

Lập luận vì sao một bộ giải mã đơn giản đủ dùng. Qua hình trực quan vùng tiếp nhận hiệu dụng (ERF, trên Cityscapes), tác giả quan sát rằng ERF của DeepLabV3+ vẫn nhỏ ngay cả ở tầng 4, trong khi bộ mã hóa MiT có chú ý mang tính cục bộ ở các tầng thấp (giống tích chập) và chú ý rất toàn cục ở tầng 4; bộ giải mã MLP gộp cả hai nên có biểu diễn bổ sung nhau chỉ với rất ít tham số. Từ đó bài báo suy ra cùng bộ giải mã này sẽ *không* hiệu quả với bộ mã hóa CNN, vì vùng tiếp nhận tổng thể bị chặn bởi tầng 4 của CNN, và kiểm chứng điều đó ở Bảng 1d (mục 5.3). Đây là giả thiết then chốt của phương pháp.

5.2.3 Họ mô hình và độ phức tạp

Bộ mã hóa MiT-B0 đến B5 có cùng kiến trúc nhưng khác độ sâu và độ rộng; bộ giải mã dùng $C = 256$ cho các mô hình thời gian thực (B0, B1) và $C = 768$ cho các mô hình còn lại. Bảng 5.1 (trích từ Bảng 1a của bài báo) cho thấy bộ giải mã rất nhỏ: chỉ 0,4M tham số ở B0, và tối đa khoảng 4% tổng số tham số ở B5.

Mô hình	Encoder (M)	Decoder (M)	GFLOPs	mIoU SS	mIoU MS
MiT-B0	3,4	0,4	8,4	37,4	38,0
MiT-B1	13,1	0,6	15,9	42,2	43,1
MiT-B2	24,2	3,3	62,4	46,5	47,5
MiT-B3	44,0	3,3	79,0	49,4	50,0
MiT-B4	60,8	3,3	95,7	50,3	51,1
MiT-B5	81,4	3,3	183,3	51,0	51,8

Bảng 5.1: Họ SegFormer trên ADE20K theo Bảng 1a của bài báo (SS: một tỉ lệ; MS: nhiều tỉ lệ).

So sánh với SETR. Bài báo nêu bốn khác biệt chính: chỉ dùng pretrain ImageNet-1K (SETR dùng ImageNet-22K); bộ mã hóa phân cấp và nhỏ hơn; không dùng positional embedding; bộ giải mã gọn hơn nhiều so với các khối nhiều tích chập 3×3 của SETR.

5.3 Thực nghiệm và kết quả của bài báo

Thiết lập. Bài báo đánh giá trên ADE20K (150 lớp, 20 210 ảnh), Cityscapes (19 lớp, 5 000 ảnh gán nhãn mịn độ phân giải cao) và COCO-Stuff (172 nhãn, 164 nghìn ảnh; kết quả nằm ở phụ lục), với metric mIoU. Mã dựa trên mmsegmentation, huấn luyện trên 8 GPU V100; bộ mã hóa được pretrain trên ImageNet-1K còn bộ giải mã khởi tạo ngẫu nhiên. Augmentation gồm đổi kích thước ngẫu nhiên 0,5–2,0, lật ngang và cắt ngẫu nhiên (512×512 cho ADE20K). AdamW,

learning rate ban đầu $6 \cdot 10^{-5}$, lịch "poly" hệ số 1,0, batch 16 (ADE20K), 160 000 vòng lặp (40 000 cho các thí nghiệm ablation). Khi đánh giá, cạnh ngắn của ảnh được đưa về kích thước cắt khi huấn luyện và giữ tỉ lệ khung hình.

Thí nghiệm 1: kích thước mô hình. Tăng cỡ bộ mã hóa cải thiện nhất quán mIoU trên cả ba tập (Bảng 5.1); B0 đạt 37,4 mIoU chỉ với 3,8M tham số, B5 đạt 51,0 (một tỉ lệ) và 51,8 (nhiều tỉ lệ).

Thí nghiệm 2: độ rộng bộ giải mã C (ADE20K). Tăng C từ 256 lên 768 nâng mIoU từ 44,9 lên 45,4 và độ cải thiện dừng lại từ khoảng 768 trở đi (45,2 ở 1024; 45,6 ở 2048 với chi phí lớn hơn nhiều; số FLOPs ở $C = 768$ trùng với MiT-B2 trong Bảng 1a, nên có thể suy ra đây là MiT-B2); vì vậy chọn $C = 256$ cho mô hình nhẹ và $C = 768$ cho mô hình còn lại.

Thí nghiệm 3: Mix-FFN và positional encoding (Cityscapes). Với positional encoding, mIoU là 77,3 ở 768×768 và 74,0 ở 1024×2048 (giảm 3,3); với Mix-FFN là 80,5 và 79,8 (giảm 0,7). Tác giả kết luận Mix-FFN vừa tốt hơn vừa bền hơn khi đổi độ phân giải.

Thí nghiệm 4: bộ mã hóa CNN so với Transformer với cùng bộ giải mã MLP (ADE20K). Bảng 5.2: các bộ mã hóa ResNet/ResNeXt cho mIoU thấp hơn rõ rệt so với MiT, và chỉ dùng đặc trưng tầng 4 của MiT-B2 (43,1) kém hơn dùng cả bốn tầng (45,4). Đây là bằng chứng bài báo dùng cho giả thiết về ERF.

Bộ mã hóa (decoder MLP)	GFLOPs	Tham số (M)	mIoU
ResNet50 (tầng 1–4)	69,2	29,0	34,7
ResNet101 (tầng 1–4)	88,7	47,9	38,7
ResNeXt101 (tầng 1–4)	127,5	86,8	39,8
MiT-B2 (chỉ tầng 4)	22,3	24,7	43,1
MiT-B2 (tầng 1–4)	62,4	27,7	45,4
MiT-B3 (tầng 1–4)	79,0	47,3	48,6

Bảng 5.2: Bảng 1d của bài báo: bộ mã hóa CNN và Transformer với bộ giải mã MLP, ADE20K.

Thí nghiệm 5: đổi bộ mã hóa và bộ giải mã (ADE20K, 160K vòng lặp). Với cùng bộ giải mã MLP, MiT-B2 (46,5) cao hơn Swin-T (43,4) khoảng 3,1 điểm với số tham số bộ mã hóa tương đương, và MiT-B5 (51,0) cao hơn ViT-L (47,7) với ít tham số bộ mã hóa hơn nhiều. Với cùng bộ mã hóa MiT, các bộ giải mã khác nhau (UperNet, MLA của SETR, MLP) cho mIoU xấp xỉ nhau (ví dụ MiT-B2: 46,5; 46,2; 46,5) nhưng bộ giải mã MLP chỉ có 3,3M tham số so với 29,7M của UperNet.

So sánh với các phương pháp khác (Bảng 2). Các mô hình thời gian thực trên ADE20K gồm trong Bảng 5.3. SegFormer-B0 đạt 37,4 mIoU với 3,8M tham số và 8,4 GFLOPs, vượt

DeepLabV3+ MobileNetV2 (34,0 mIoU, 15,4M tham số, 69,4 GFLOPs) 3,4 điểm đồng thời nhanh hơn 7,4 FPS. Ở nhóm hiệu năng cao, SegFormer-B5 đạt 51,8 mIoU (nhiều tỉ lệ), hơn SETR (50,2) và nhỏ hơn khoảng 4 lần. Trên Cityscapes, B0 đạt 76,2 mIoU ở 15,2 FPS (cạnh ngắn 1024), hơn DeepLabV3+ MobileNetV2 (75,2); B5 đạt 84,0 trên tập validation và 82,2 trên tập test (83,1 nếu thêm pretrain Mapillary).

Phương pháp	Encoder	Tham số (M)	GFLOPs	FPS	mIoU
FCN	MobileNetV2	9,8	39,6	64,4	19,7
PSPNet	MobileNetV2	13,7	52,9	57,7	29,6
DeepLabV3+	MobileNetV2	15,4	69,4	43,1	34,0
SegFormer	MiT-B0	3,8	8,4	50,5	37,4

Bảng 5.3: Các phương pháp thời gian thực trên ADE20K trong Bảng 2 của bài báo.

Thí nghiệm 6: độ bền trước nhiễu (Cityscapes-C). Bài báo mở rộng tập validation Cityscapes với 16 loại biến đổi nhân tạo (nhiễu, mờ, thời tiết, kỹ thuật số) theo [5] và so sánh SegFormer-B5 với DeepLabV3+ và các phương pháp khác. SegFormer-B5 vượt các mô hình dựa trên CNN và nhìn chung vượt SETR (dùng DeiT), trừ một loại nhiễu (tuyết); các cải thiện tương đối được nêu là tới 588% với nhiễu Gauss và tới 295% với tuyết so với phương pháp CNN.

Kết luận của tác giả. Thiết kế đơn giản gồm bộ mã hóa phân cấp không positional encoding và bộ giải mã All-MLP đạt đồng thời độ chính xác, hiệu quả và độ bền cao, và có thể làm baseline vững cho phân đoạn ngữ nghĩa.

5.4 Hạn chế của bài báo

Tác giả tự nêu một hạn chế: ngay cả mô hình nhẹ nhất vẫn có thể còn quá nặng cho một số thiết bị biên, nên huấn luyện độ chính xác hỗn hợp, cắt tỉa, thiết kế attention thân thiện với phần cứng và năng lượng tiêu thụ được để lại cho công việc sau. Ngoài ra, nhóm nhận thấy các điểm sau (bản đầu; sẽ phân tích sâu và đối chiếu với kết quả ở mốc A2.2):

- **Không có sai số giữa các lần chạy.** Bài báo khai báo không có error bar và cho rằng mIoU "khá ổn định". Các chênh lệch nhỏ (ví dụ giữa các bộ giải mã trong Bảng 3 chỉ vài phần mười điểm) vì vậy không thể đánh giá độ tin cậy. Nhóm chạy nhiều seed để bù phần này.
- **Phép so sánh Mix-FFN và positional encoding chưa được kiểm soát chặt.** Hai cấu hình được suy luận ở hai độ phân giải với hai kiểu suy luận khác nhau (cửa sổ trượt

768×768 và toàn ảnh 1024×2048), nên độ phân giải và quy trình đánh giá bị lẫn vào nhau. Nhóm dự định đánh giá độ bền theo độ phân giải với quy trình thống nhất nếu còn thời gian.

- **Bằng chứng về độ bền chỉ có cho mô hình lớn nhất.** Bảng Cityscapes-C chỉ có SegFormer-B5; không rõ liệu B0 (mô hình nhẹ mà nhóm dùng) có giữ được ưu thế như vậy.
- **Phần lớn ablation ở lịch ngắn.** Các thí nghiệm ablation dùng 40 000 vòng lặp thay vì 160 000, nên thứ hạng có thể thay đổi khi huấn luyện lâu hơn.
- **Số liệu của đối thủ lấy từ nguồn khác.** Nhiều số liệu trong Bảng 2 của các phương pháp khác được lấy từ các công trình đó với công thức huấn luyện riêng, nên phép so sánh không hoàn toàn cùng điều kiện; Bảng 2 cũng không nêu rõ GPU và độ phân giải dùng để đo FPS và FLOPs ở phần chính.
- **Chi phí huấn luyện lớn.** Lịch 160 000 vòng lặp trên 8 GPU V100 khó tái hiện ở quy mô nhỏ; bài báo không phân tích độ nhạy với lịch ngắn hay với việc huấn luyện từ đầu, hầu hết kết quả dựa trên encoder pretrain ImageNet-1K.
- **Không nhất quán nhỏ về số liệu.** Số tham số của SegFormer-B0 được nêu là 3,7M ở Hình 1 và 3,8M ở Bảng 2 (có thể do bản phân loại ImageNet có thêm lớp phân loại 1000 lớp); nhóm báo cáo số tham số tự đếm.
- **Thiếu phân tích lỗi.** Phần chính của bản NeurIPS không có phân tích định tính các trường hợp thất bại (lớp nào, loại đối tượng nào khó), và hình ERF chỉ trực quan hóa định tính trên một tập (Cityscapes) với DeepLabV3+.

5.5 Liên hệ với thí nghiệm của nhóm

Thí nghiệm của nhóm tái hiện phần thời gian thực của Bảng 2 (SegFormer-B0 so với DeepLabV3+ MobileNetV2 trên ADE20K) với lịch huấn luyện ngắn hơn khoảng 6,7 lần, bổ sung nhiều seed cho các cấu hình pretrained, và thêm các biến thể huấn luyện từ đầu để kiểm tra mức phụ thuộc vào pretrain ImageNet-1K mà bài báo mặc định. Nhóm cũng đối chiếu số tham số và GMACs tự đo với Bảng 2 (khớp: 3,75M và 8,39G; 15,35M và 69,44G) và kiểm chứng mã đánh giá bằng checkpoint công bố (36,70 so với 37,4 mIoU).

Chương 6

KẾT QUẢ THỰC NGHIỆM

6.1 Hiệu quả của các phương pháp

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

6.1.1 Bảng và biểu đồ tổng hợp

6.1.2 So sánh CNN và Transformer

6.1.3 So sánh scratch và pretrained

6.1.4 Ảnh hưởng của learning rate và độ ổn định giữa các seed

6.1.5 IoU theo lớp và tần suất lớp

6.2 Diễn biến quá trình huấn luyện

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

6.2.1 Learning curves

6.2.2 Kết quả dò learning rate

6.3 Kết quả trực quan

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

6.3.1 Ví dụ đầu ra và đối chiếu ground truth

6.4 Chi phí tính toán

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

Chương 7

PHÂN TÍCH VÀ THẢO LUẬN

7.1 So sánh các phương pháp hoặc biến thể

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

7.1.1 Khác biệt về hiệu quả và các yếu tố ảnh hưởng

7.1.2 Ưu điểm và hạn chế của từng phương pháp

7.2 Phân tích các trường hợp thành công và thất bại

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

7.2.1 Trường hợp hoạt động tốt

7.2.2 Failure cases và nguyên nhân khả dĩ

7.3 Đối chiếu với bài báo khoa học

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

7.4 Hạn chế của thực nghiệm

Các sai khác so với bài báo và các hạn chế của thiết kế thực nghiệm đã biết trước khi chạy như sau (kèm tác động dự kiến lên cách đọc kết quả); phần đánh giá định lượng sẽ bổ sung ở mốc A2.2.

- **Lịch huấn luyện ngắn (15% so với bài báo).** 24 000 so với 160 000 vòng lặp: mIoU tuyệt đối sẽ thấp hơn; các lượt huấn luyện từ đầu hội tụ chưa đủ nên khoảng cách pretrained so với from-scratch có thể bị phóng đại so với lịch dài.
- **Ít dữ liệu huấn luyện hơn khoảng 5%.** Tập train có 19 210 ảnh vì 1 000 ảnh được dùng làm tập dev.
- **Thống kê BatchNorm trên 8 ảnh.** DeepLabV3+ dùng tích lũy gradient 2×8 thay vì SyncBN trên 16 ảnh, có thể ảnh hưởng đến baseline CNN.
- **Trọng số ImageNet khác nhau.** MobileNetV2 dùng trọng số của torchvision, khác trọng số của mmcls mà mmsegmentation dùng.
- **Độ chính xác fp16.** Bài báo không nêu dùng độ chính xác hỗn hợp; nhóm dùng fp16 để vừa bộ nhớ và rút ngắn thời gian.
- **Công thức tối ưu khác nhau giữa hai mô hình.** Kiến trúc và công thức tối ưu bị lẫn vào nhau (mục 4.2.1); learning rate được dò ở mức hạn chế (ba giá trị, 3 epoch, một seed) nên chưa chắc đã tối ưu cho lịch 20 epoch.
- **Số seed ít.** Ba seed cho cấu hình pretrained và chỉ một seed cho cấu hình từ đầu; độ lệch chuẩn ước lượng từ ba giá trị chỉ mang tính tham khảo.
- **Một loại GPU, một tỉ lệ đánh giá.** Số liệu thời gian chỉ có giá trị tương đối trên Tesla T4; không đánh giá đa tỉ lệ hay lật ảnh như một số số liệu trong bài báo.
- **Phạm vi dữ liệu.** Chỉ ADE20K; kết quả có thể không khái quát hóa sang Cityscapes hay COCO-Stuff.

Chương 8

KẾT LUẬN

8.1 Tổng kết kết quả

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

8.2 Hướng phát triển

Phần này sẽ được hoàn thiện ở mốc A2.2 (16/11/2026), sau khi các lượt huấn luyện hoàn tất.

TÀI LIỆU THAM KHẢO

[1] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. Rethinking atrous convolution for semantic image segmentation. *arXiv preprint arXiv:1706.05587*, 2017.

[2] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European Conference on Computer Vision*, 2018.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009.

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[5] Christoph Kamann and Carsten Rother. Benchmarking the robustness of semantic segmentation models. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2020.

[6] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[8] MMSegmentation Contributors. MMSegmentation: OpenMMLab semantic segmentation toolbox and benchmark. `https://github.com/open-mmlab/mmsegmentation`, 2020.

[9] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[10] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[12] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

[13] Thomas Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[14] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems*, volume 34, 2021. arXiv:2105.15203.

[15] Sixiao Zheng, Jiachen Lu, Hengshuang Zhao, Xiatian Zhu, Zekun Luo, Yabiao Wang, Yanwei Fu, Jianfeng Feng, Tao Xiang, Philip H. S. Torr, and Li Zhang. Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2021.

[16] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ADE20K dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

Phụ lục A

KẾT QUẢ THỰC NGHIỆM BỔ SUNG

Các bảng và thí nghiệm bổ sung (kết quả dò learning rate đầy đủ, IoU của từng lớp, ma trận nhầm lẫn) sẽ được đưa vào ở mốc A2.2 (16/11/2026).

Phụ lục B

KẾT QUẢ TRỰC QUAN BỔ SUNG

Các hình dự đoán, nhãn thật và ví dụ bổ sung sẽ được đưa vào ở mốc A2.2 (16/11/2026).

Phụ lục C

CẤU HÌNH VÀ TÁI LẬP

C.1 Dữ liệu

Tệp `ADEChallengeData2016.zip`: 967 382 037 byte, sha256 `7ff1be44964418441f542a7cc1e1a650e7dc0fc275f5d23252bc9bbdbc977b29`. Ban tổ chức không công bố mã băm, đây là mã băm của tệp mà nhóm đã tải về (25 giây từ `http://data.csail.mit.edu/places/ADEchallenge/ADEChallengeData2016.zip`).

C.2 Cấu hình và lệnh chạy

Tệp cấu hình của từng lượt chạy, mã băm `split.json`*, mã commit và các lệnh chạy lại (huấn luyện, dò learning rate, đánh giá, sinh bảng và hình) sẽ được liệt kê ở mốc A2.2 khi mã huấn luyện được đưa lên kho.*